<Lab Meeting>

AutoML

2015.05.19

Jungtaek Kim
(jtkim@postech.edu)
Machine Learning Group,
Department of Computer Science and Engineering,
POSTECH,
77 Cheongam-ro, Nam-gu, Pohang-si,
Gyungsangbuk-do, Republic of Korea

Machine Learning Group

Contents

- AutoML
- Bayesian Optimization
- Implementation of AutoML
- Reference

AutoML

AutoML

- Machine learning success in recent years crucially relies on human machine learning experts.
- They select appropriate features, workflows, machine learning paradigms, algorithms, and their hyperparameters manually.
- Off-the-shelf machine learning methods applied easily are used without expert knowledge.

[1] AutoML workshop @ ICML'15 Official Site. https://sites.google.com/site/automlwsicml15/

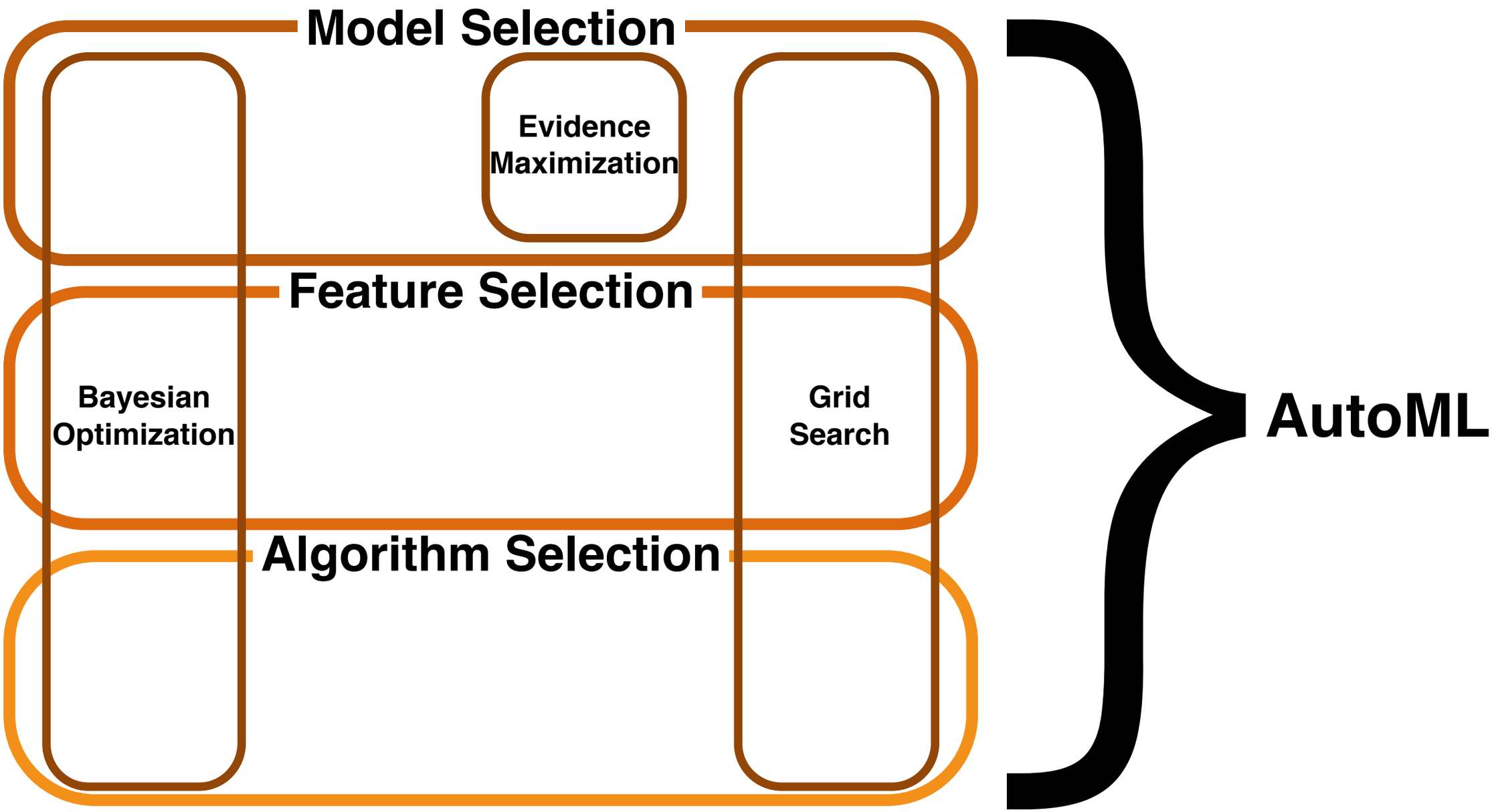
Real Issues

- One of an automatic approach to solve a combined algorithm selection and hyperparameter optimization (CASH) problem, Auto-WEKA handles 39 WEKA classification algorithms (27 ‘base’ classifiers, 10 meta methods and 2 ensemble classifiers), 3 feature search methods, 8 feature evaluators and their respective number of sub subparameters.
- Caffe deep neural network software considers 81 hyperparameters (9 network parameters, 12 parameters for each of up to 6 layers).

[2] C. Thornton, F. Hutter, H. H. Hoos, K. Leyton-Brown. Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms. KDD, 2013.

[3] T. Domhan, T. Springenberg, F. Hutter. Extrapolating Learning Curves of Deep Neural Networks. ICML 2014 AutoML Workshop, 2014.

Big Picture of AutoML



Preliminaries

- A model parameter learning is

$$\underset{\theta \in \Theta}{\operatorname{argmin}} \, \mathcal{L}(f(\theta; \lambda_i, F_j, A_i, \{\mathbf{x}_k, y_k\}_{k=1}^{n}))$$

where $\mathcal{L}$ is a loss function, f is a predictive model, θ is a parameter vector, λ_i is the hyperparameter vector of the chosen algorithm A_i, and F_j is the chosen feature vector.

AutoML

- Each problem optimizes parameter, hyperparameter, feature, and model.
  - Model selection

$$\underset{\theta \in \Theta, \lambda_i \in \Lambda}{\operatorname{argmin}} \mathcal{L}(f(\theta, \lambda_i; F_j, A_i, \{\mathbf{x}_k, y_k\}_{k=1}^n))$$

  - Feature selection

$$\underset{\theta \in \Theta, \lambda_i \in \Lambda, F_j}{\operatorname{argmin}} \mathcal{L}(f(\theta, \lambda_i, F_j; A_i, \{\mathbf{x}_k, y_k\}_{k=1}^n))$$

  - Algorithm selection

$$\underset{\theta \in \Theta, \lambda_i \in \Lambda, F_j, A_i \in \mathcal{A}}{\operatorname{argmin}} \mathcal{L}(f(\theta, \lambda_i, F_j, A_i; \{\mathbf{x}_k, y_k\}_{k=1}^n))$$

AutoML Details

- It is all aspects of automating the machine learning process.
- It includes:
  - Representation learning and automatic feature extraction/construction.
  - Automatic generation and reuse of workflows.
  - Meta learning and transfer learning.
  - Automatic acquisition of new data (active learning, experimental design).
  - Automatic creation of appropriately sized and stratified train, validation, and test sets.
  - Automatic selection of algorithms to satisfy time / space / power constraints at traintime or at runtime.

[4] I. Guyon, et al. Design of the 2015 ChaLearn AutoML Challenge. 2015.

The reasons why AutoML is hard

- Different data distributions
- Different tasks
- Different scoring metrics
- Class balance
- Sparsity
- Missing values
- Categorical variables
- Irrelevant variables
- Number of training examples
- Number of variables/features
- Aspect ratio of the training data matrix

[4] I. Guyon, et al. Design of the 2015 ChaLearn AutoML Challenge. 2015.

Bayesian Optimization

Hyperparameter Optimization

- Hyperparameter optimization can suggest to tune model parameters, hyperparameters, feature and model.
- The issue of AutoML can be represented to the perspective to optimize such parameters.
- Bayesian optimization is a framework for the optimization of expensive blackbox functions that combines prior assumptions about the shape of a function with evidence gathered by evaluating the function at various points.

[5] J. Snoek, H. Larochelle, R. P. Adams. Practical Bayesian Optimization of Machine Learning Algorithms. NIPS, 2012.

[6] F. Hutter. Bayesian Optimization for More Automatic Machine Learning. ECAI 2014 Meta-Learning and Algorithm Selection Workshop, 2014.

Bayesian Optimization

- Posterior distribution is

$$P(f|\mathcal{D}_{1:t}) \propto P(\mathcal{D}_{1:t}|f)P(f)$$

where $f(\mathbf{x}_i)$ is the observations of the objective function at $\mathbf{x}_i$.

- As observations $\mathcal{D}_{1:t} = \{\mathbf{x}_{1:t}, f(\mathbf{x}_{1:t})\}$ are accumulated, the prior distribution is combined with the likelihood function $P(\mathcal{D}_{1:t}|f)$.

[7] E. Brochu, V. M. Cora, N. de Freitas. A tutorial on Bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning. Technical Report UBC TR-2009-23 and arXiv:1012.2599v1, 2009.

Priors over Functions

- A GP is a distribution over functions, completely specified by its mean function, m and covariance function, k :

$$f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$$

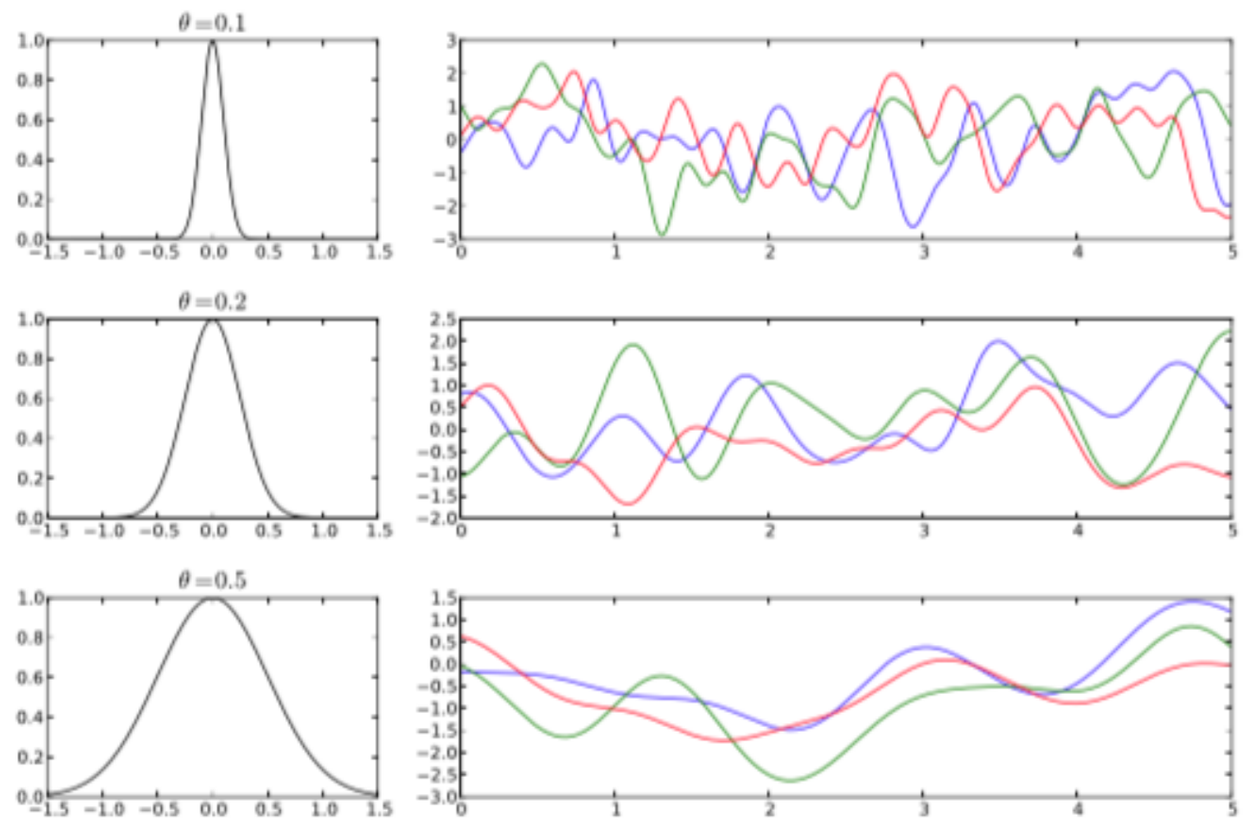


Figure 3: *The effect of changing the kernel hyperparameters. Shown are squared exponential kernels with* $\theta = 0.1, 0.2, 0.5$. *On the left is the function* $k(0, \mathbf{x})$. *On the right are some one-dimensional functions sampled from a GP with the hyperparameter value.*

Bayesian Optimization with Gaussian Process Priors

- As a result, there are two major choices that must be made when performing Bayesian optimization,
  - (i) Surrogate function (Response surface) - It makes a posterior based on a prior over functions with the Gaussian process prior, due to its flexibility and tractability.
  - (ii) Acquisition function - It is used to construct a utility function from the model posterior, allowing us to determine the next point to evaluate.

[5] J. Snoek, H. Larochelle, R. P. Adams. Practical Bayesian Optimization of Machine Learning Algorithms. NIPS, 2012.

[7] E. Brochu, V. M. Cora, N. de Freitas. A tutorial on Bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning. Technical Report UBC TR-2009-23 and arXiv:1012.2599v1, 2009.

Bayesian Optimization with Gaussian Process Priors

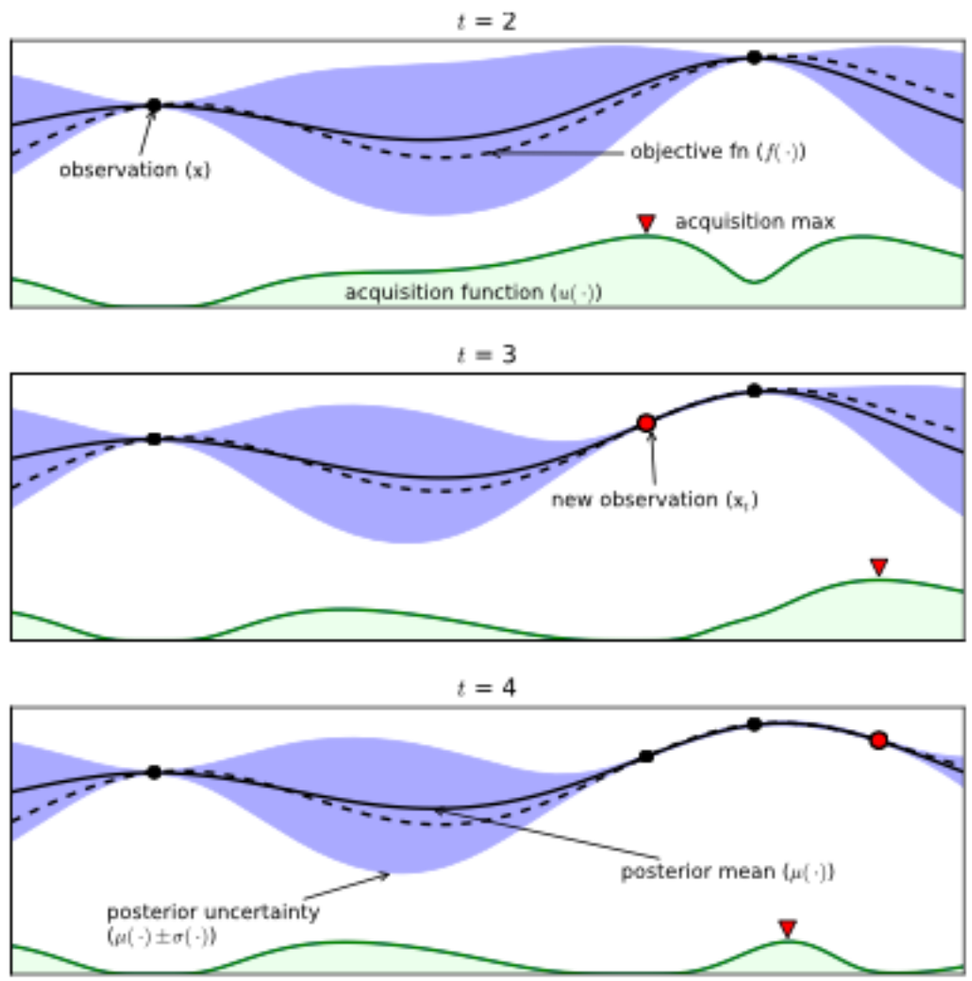


Figure 1: *An example of using Bayesian optimization on a toy 1D design problem. The figures show a Gaussian process (GP) approximation of the objective function over four iterations of sampled values of the objective function. The figure also shows the acquisition function in the lower shaded plots. The acquisition is high where the GP predicts a high objective (exploitation) and where the prediction uncertainty is high (exploration)—areas with both attributes are sampled first. Note that the area on the far left remains unsampled, as while it has high uncertainty, it is (correctly) predicted to offer little improvement over the highest observation.*

[7] E. Brochu, V. M. Cora, N. de Freitas. A tutorial on Bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning. Technical Report UBC TR-2009-23 and arXiv: 1012.2599v1, 2009.

Acquisition Functions for Bayesian Optimization

- Preliminaries
  - The function $f(x)$ is drawn from a Gaussian process prior.
  - The observations are of the form $\{\mathbf{x}_n, y_n\}_{n=1}^N$, where $y_n \sim \mathcal{N}(f(\mathbf{x}_n), \nu)$ and ν is the variance of noise.
  - The acquisition function which is induced by the prior and data is $a : \mathcal{X} \to \mathbb{R}^+$.
  - The predictive mean function $\mu(\mathbf{x}; \{\mathbf{x}_n, y_n\}, \theta)$ and the predictive variance function $\sigma^2(\mathbf{x}; \{\mathbf{x}_n, y_n\}, \theta)$.
  - The best current value is $\mathbf{x}_{\text{best}} = \text{argmin}_{\mathbf{x}_n} f(\mathbf{x}_n)$.
  - The cumulative distribution function is $\Phi(\cdot)$.

[5] J. Snoek, H. Larochelle, R. P. Adams. Practical Bayesian Optimization of Machine Learning Algorithms. NIPS, 2012.

Acquisition Functions for Bayesian Optimization

- Probability of Improvement (PI)

$$a_{\mathsf{PI}}(\mathbf{x}; \{\mathbf{x}_n, y_n\}, \theta) = \Phi(\gamma(\mathbf{x})), \quad \gamma(\mathbf{x}) = \frac{f(\mathbf{x}_{\mathsf{best}}) - \mu(\mathbf{x}; \{\mathbf{x}_n, y_n\}, \theta)}{\sigma(\mathbf{x}; \{\mathbf{x}_n, y_n\}, \theta)}$$

- Expected Improvement (EI)

$$a_{\mathsf{EI}}(\mathbf{x}; \{\mathbf{x}_n, y_n\}, \theta) = \sigma(\mathbf{x}; \{\mathbf{x}_n, y_n\}, \theta)(\gamma(\mathbf{x})\Phi(\gamma(\mathbf{x})) + \mathcal{N}(\gamma(\mathbf{x}); 0, 1))$$

- GP Upper Confidence Bound (GP-UCB)

$$a_{\mathsf{UCB}}(\mathbf{x}; \{\mathbf{x}_n, y_n\}, \theta) = \mu(\mathbf{x}; \{\mathbf{x}_n, y_n\}, \theta) + \kappa\sigma(\mathbf{x}; \{\mathbf{x}_n, y_n\}, \theta)$$

[9] H. J. Kushner. A new method of locating the maximum of an arbitrary multipeak curve in the presence of noise. Journal of Basic Engineering, 86:97-106, 1964.
[10] J. Močkus, V. Tiesis, and A. Žilinskas. The application of Bayesian methods for seeking the extremum. Towards Global Optimization, 2:117-129, 1978.
[11] D. R. Jones, M. Schonlau, and W. J. Welch. Efficient global optimization of of expensive black-box functions. Journal of Global Optimization, 13(4):455-492, 1998.
[12] N. Srinivas, A. Krause, S. M. Kakade, and M. Seeger. Gaussian process optimization in the bandit setting: No regret and experimental design. ICML, 2010.

Acquisition Functions for Bayesian Optimization

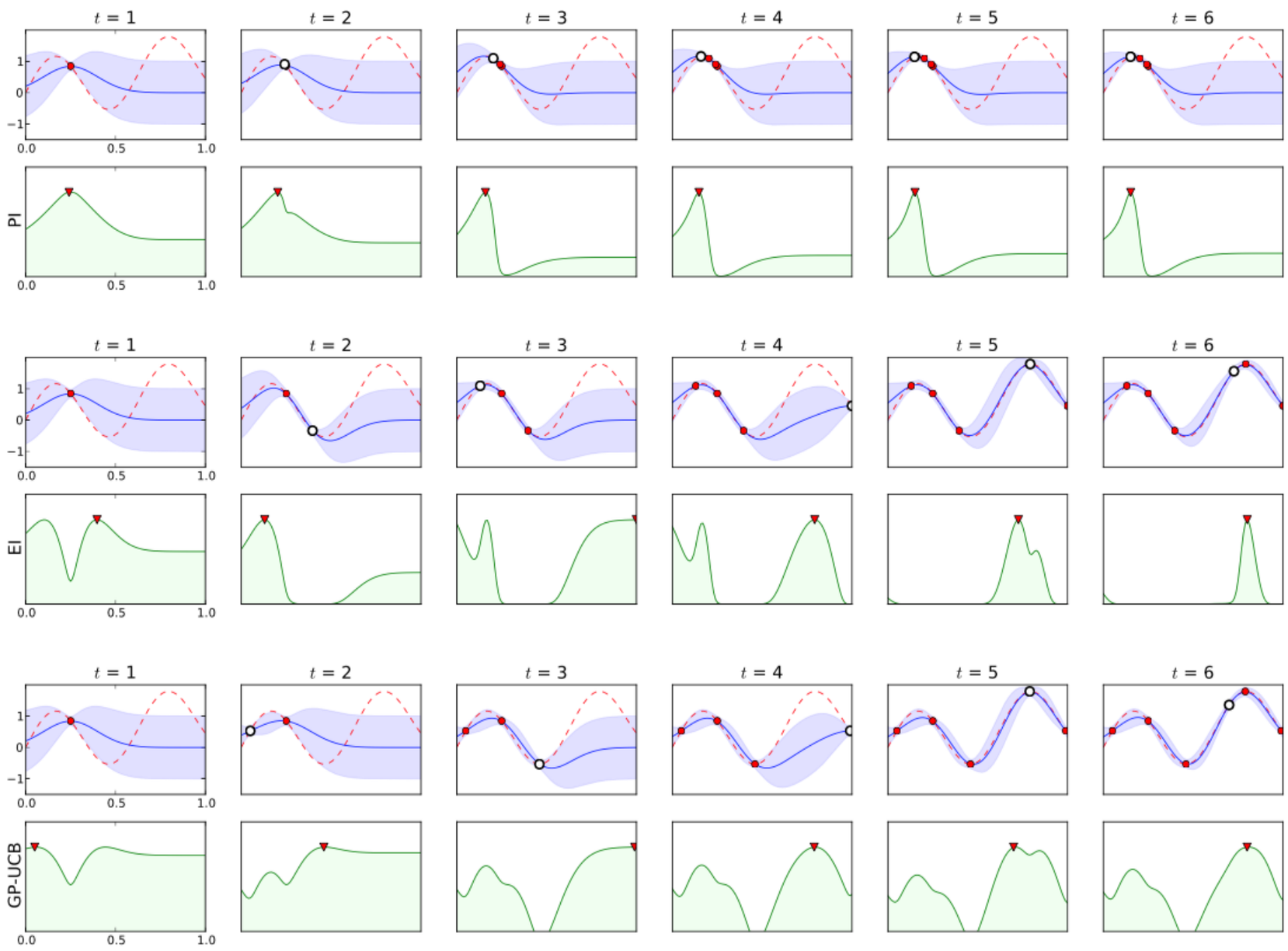


Figure 7: *Comparison of probability of improvement (top), expected improvement (middle) and upper confidence bound (bottom) acquisition functions on a toy 1D problem. In the upper rows, the objective function is shown with the dotted red line, the solid blue line is the GP posterior mean. In the lower rows, the respective infill functions are shown, with a star denoting the maximum. The optimizations are initialized with the same two points, but quickly follow different sampling trajectories. In particular, note that the greedy* EI *algorithm ignores the region around* $x = 0.4$ *once it is determined there is minimal chance of improvement, while* GP-UCB *continues to explore.*

[7] E. Brochu, V. M. Cora, N. de Freitas. A tutorial on Bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning. Technical Report UBC TR-2009-23 and arXiv: 1012.2599v1, 2009.

Acquisition Function for EI Criterion

- The predictive distribution of the Gaussian process enables to balance the trade-off of exploiting and exploring.
- The most commonly used acquisition function of expected positive improvement is

$$a_{EI}(\theta, \mathcal{M}) = \int_{-\infty}^{\infty} max(y^* - y, 0) p_{\mathcal{M}}(y|\theta) d_y.$$

- All of SMAC, Spearmint, and TPE use the EI criterion.

[7] E. Brochu, V. M. Cora, N. de Freitas. A tutorial on Bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning. Technical Report UBC TR-2009-23 and arXiv:1012.2599v1, 2009.

[13] M. Feurer, J. T. Springenberg, and F. Hutter. Using Meta-Learning to Initialize Bayesian Optimization of Hyperparameters. ECAI workshop on Metalearning and Algorithm Selection (MetaSel), 2014.

Sequential Model-based Bayesian Optimization (SMBO)

- SMBO is one of a promising approach and method of Bayesian optimization.
- SMBO can work explicitly with both categorical and continuous hyperparameters and exploit hierarchical structure stemming from conditional parameters.
- SMBO can be applied to general algorithm configuration problems with many categorical parameters and sets of benchmark instances.

[2] C. Thornton, F. Hutter, H. H. Hoos, K. Leyton-Brown. Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms. KDD, 2013.
[14] F. Hutter, H. H. Hoos, and K. Leyton-Brown. Sequential Model-Based Optimization for General Algorithm Configuration. Learning and Intelligent Optimization, 507-523, 2011.

Sequential Model-based Bayesian Optimization (SMBO)

- Hutter el al. have developed SMBO strategies for the configuration of satisfiability and MIP solvers using random forests.
- It iterates the following three phases:
  - (i) fit a probabilistic model $\mathcal{M}$ to <input, output> pairs collected so far.
  - (ii) use the probabilistic model $\mathcal{M}$ to select a promising input θ to evaluate next by quantifying the desirability of obtaining through an acquisition function $a(\theta, \mathcal{M})$.
  - (iii) evaluate the function at the new input θ.

[5] J. Snoek, H. Larochelle, R. P. Adams. Practical Bayesian Optimization of Machine Learning Algorithms. NIPS, 2012.
[14] F. Hutter, H. H. Hoos, and K. Leyton-Brown. Sequential Model-Based Optimization for General Algorithm Configuration. Learning and Intelligent Optimization, 507-523, 2011.
[13] M. Feurer, J. T. Springenberg, and F. Hutter. Using Meta-Learning to Initialize Bayesian Optimization of Hyperparameters. ECAI workshop on Metalearning and Algorithm Selection (MetaSel), 2014.

Sequential Model-based Bayesian Optimization (SMBO)

Algorithm 1: Generic Sequential Model-based Optimization.
SMBO($f^D, T, \boldsymbol{\Theta}, \boldsymbol{\theta}_{1:t}$)

Input: Target function f^D; limit T; hyperparameter space $\boldsymbol{\Theta}$;
initial design $\boldsymbol{\theta}_{1:t} = \langle \boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_t \rangle$
Result: Best hyperparameter configuration $\boldsymbol{\theta}^*$ found

for $i \leftarrow 1$ **to** t **do** $y_i \leftarrow$ Evaluate $f^D(\boldsymbol{\theta}_i)$
for $j \leftarrow t+1$ **to** T **do**
  $\mathcal{M} \leftarrow$ fit model on performance data $\langle \boldsymbol{\theta}_i, y_i \rangle_{i=1}^{j-1}$
  Select $\boldsymbol{\theta}_j \in \arg\max_{\boldsymbol{\theta} \in \boldsymbol{\Theta}} a(\boldsymbol{\theta}, \mathcal{M})$
  $y_j \leftarrow$ Evaluate $f^D(\boldsymbol{\theta}_j)$
return $\boldsymbol{\theta}^* \in \arg\min_{\boldsymbol{\theta}_j \in \{\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_T\}} y_j$

[13] M. Feurer, J. T. Springenberg, and F. Hutter. Using Meta-Learning to Initialize Bayesian Optimization of Hyperparameters. ECAI workshop on Metalearning and Algorithm Selection (MetaSel), 2014.

Sequential Model-based Algorithm Configuration (SMAC)

- SMAC handles conditional parameters by instantiating inactive condition parameters to use random forests.
- It obtains a predictive mean and variance of $p(c|\lambda)$ as frequentist estimates over the predictions of its individual tree for λ ; it then

$$p_{\mathcal{M}}(c|\lambda) = \mathcal{N}(\mu_\lambda, \sigma_\lambda^2).$$

- A key idea in SMAC is to make progressively better estimates o this mean by evaluating these terms one at a time, thus trading off accuracy against computational cost.

[2] C. Thornton, F. Hutter, H. H. Hoos, K. Leyton-Brown. Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms. KDD, 2013.

[14] F. Hutter, H. H. Hoos, and K. Leyton-Brown. Sequential Model-Based Optimization for General Algorithm Configuration. Learning and Intelligent Optimization, 507-523, 2011.

Sequential Model-based Algorithm Configuration (SMAC)

- The expectation can be computed by the closed-form expression,

$$\mathbb{E}[I_{c_{min}}(\lambda)] = \sigma_\lambda \cdot [u \cdot \Phi(u) + \varphi(u)],$$

- where $u = \dfrac{c_{min} - \mu_\lambda}{\sigma_\lambda}$, and φ and Φ denote the probability density function and cumulative distribution function of a standard normal distribution, respectively.

[2] C. Thornton, F. Hutter, H. H. Hoos, K. Leyton-Brown. Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms. KDD, 2013.

[14] F. Hutter, H. H. Hoos, and K. Leyton-Brown. Sequential Model-Based Optimization for General Algorithm Configuration. Learning and Intelligent Optimization, 507-523, 2011.

Tree-structured Parzen Estimator Approach (TPE)

- Hyperparameter optimization tasks need high dimensions and small fitness evaluation budgets.
- TPE strategy models $p(c)$ and $p(\lambda|c)$.
- It makes the following replacements:
  - uniform -> truncated Gaussian mixture
  - log-uniform -> exponentiated truncated Gaussian mixture
  - categorical -> re-weighted categorical
- TPE assumes independence for hyperparameters that do not appear together along any path from the tree's root to one of its leaves.

[15] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl. Algorithms for Hyper-Parameter Optimization. NIPS, 2011.

[2] C. Thornton, F. Hutter, H. H. Hoos, K. Leyton-Brown. Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms. KDD, 2013.

Tree-structured Parzen Estimator Approach (TPE)

- $p(\lambda|c)$ is modeled as one of two density estimates:

$$p(\lambda|c) = \begin{cases} l(\lambda) & \text{if } c < c^* \\ g(\lambda) & \text{if } c \geq c^* \end{cases}$$

where c^* is chosen as the γ-quantile of the losses TPE.

- Intuitively, probabilistic density estimators are $l(.)$ for hyperparameters that appear to do 'well', and $g(.)$ for hyperparameters that appear 'poor'.
- EI can be computed in closed-form:

$$\mathbb{E}[I_{c_{min}}(\lambda)] \propto (\gamma + \frac{g(\lambda)}{l(\lambda)}(1-\gamma))^{-1}.$$

[15] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl. Algorithms for Hyper-Parameter Optimization. NIPS, 2011.

[2] C. Thornton, F. Hutter, H. H. Hoos, K. Leyton-Brown. Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms. KDD, 2013.

Implementation of AutoML

Auto-WEKA

- Auto-WEKA is to solve a combined algorithm selection and hyperparameter optimization (CASH) problem.
- It handles 39 WEKA classification algorithms (27 ‘base’ classifiers, 10 meta methods and 2 ensemble classifiers), 3 feature search methods, 8 feature evaluators and their respective number of sub subparameters.
- It shows classification performance often much better than using standard selection/ hyperparameter optimization methods for 21 popular datasets.

[2] C. Thornton, F. Hutter, H. H. Hoos, K. Leyton-Brown. Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms. KDD, 2013.

Preliminaries

- Algorithm selection (Model selection in [2])

$$A^* \in \underset{A \in \mathcal{A}}{\operatorname{argmin}} \frac{1}{k} \sum_{i=1}^{k} \mathcal{L}(A, \mathcal{D}_{train}^{(i)}, \mathcal{D}_{valid}^{(i)}),$$

where $\mathcal{L}(A, \mathcal{D}_{train}^{(i)}, \mathcal{D}_{valid}^{(i)})$ is the loss. k-fold cross-validation is used.

- Model selection (Hyperparameter optimization in [2])

$$\lambda^* = \underset{\lambda \in \mathbf{\Lambda}}{\operatorname{argmin}} \frac{1}{k} \sum_{i=1}^{k} \mathcal{L}(A_\lambda, \mathcal{D}_{train}^{(i)}, \mathcal{D}_{valid}^{(i)}),$$

where the hyperparameter space $\mathbf{\Lambda}$ is a subset of the crossproduct of these domains: $\mathbf{\Lambda} \subset \Lambda_1 \times \ldots \times \Lambda_n$.

[2] C. Thornton, F. Hutter, H. H. Hoos, K. Leyton-Brown. Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms. KDD, 2013.

Combined Algorithm Selection and Hyperparameter Optimization (CASH)

- Given a set of algorithms $\mathcal{A} = \{A^{(1)}, \ldots, A^{(k)}\}$ with associated hyperparameter spaces $\mathbf{\Lambda}^{(1)}, \ldots, \mathbf{\Lambda}^{(k)}$,

$$A^*_{\lambda^*} \in \underset{A^{(j)} \in \mathcal{A}, \lambda \in \mathbf{\Lambda}^{(j)}}{\operatorname{argmin}} \frac{1}{k} \sum_{i=1}^{k} \mathcal{L}(A^{(j)}_{\lambda}, \mathcal{D}^{(i)}_{train}, \mathcal{D}^{(i)}_{valid}),$$

where parameter space with a single combined hierarchical hyperparameter $\mathbf{\Lambda} = \mathbf{\Lambda}^{(1)} \cup \ldots \cup \mathbf{\Lambda}^{(k)} \cup \{\lambda_r\}$ is given.

- λ_r is a new root-level hyperparameter that selects between algorithms $A^{(1)}, \ldots, A^{(k)}$.
- To solve the CASH problem, Bayesian optimization is applied.

[2] C. Thornton, F. Hutter, H. H. Hoos, K. Leyton-Brown. Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms. KDD, 2013.

Auto-WEKA

Table 1: Classifiers in Auto-WEKA: Classifiers marked with * are meta-methods, which in addition to their own parameters take one 'base' classifier and its parameters. Classifiers marked with + are ensemble methods that take as input up to 5 'base' classifiers and their parameters. *Categorical* and *Numeric* refer to the number of hyperparameters of each kind for each classifier.

Classifier	Categorical	Numeric
Bayes Net	2	0
Naive Bayes	2	0
Naive Bayes Multinomial	0	0
Gaussian Process	3	6
Linear Regression	2	1
Logistic Regression	0	1
Single-Layer Perceptron	5	2
Stochastic Gradient Descent	3	2
SVM	4	6
Simple Linear Regression	0	0
Simple Logistic Regression	2	1
Voted Perceptron	1	2
KNN	4	1
K-Star	2	1
Decision Table	4	0
RIPPER	3	1
M5 Rules	3	1
1-R	0	1
PART	2	2
0-R	0	0
Decision Stump	0	0
C4.5 Decision Tree	6	2
Logistic Model Tree	5	2
M5 Tree	3	1
Random Forest	2	3
Random Tree	4	4
REP Tree	2	3
Locally Weighted Learning*	3	0
AdaBoost M1*	2	2
Additive Regression*	1	2
Attribute Selected*	2	0
Bagging*	1	2
Classification via Regression*	0	0
LogitBoost*	4	4
MultiClass Classifier*	3	0
Random Committee*	0	1
Random Subspace*	0	2
Voting+	1	0
Stacking+	0	0

[2] C. Thornton, F. Hutter, H. H. Hoos, K. Leyton-Brown. Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms. KDD, 2013.

Auto-WEKA

Table 2: Feature Search/Evaluator methods in Auto-WEKA: Methods marked with * are search methods, which require one feature evaluator that is used to determine the importance of a feature. *Categorical* and *Numeric* refer to the number of hyperparameters of each method.

Feature Method	Categorical	Numeric
Best First*	1	1
Greedy Stepwise*	3	2
Ranker*	0	1
CFS Subset Eval	2	0
Pearson Correlation Eval	0	0
Gain Ratio Eval	0	0
Info Gain Eval	2	0
1-R Eval	1	2
Principal Components Eval	2	2
RELIEF Eval	1	2
Symmetrical Uncertainty Eval	1	0

[2] C. Thornton, F. Hutter, H. H. Hoos, K. Leyton-Brown. Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms. KDD, 2013.

Auto-WEKA

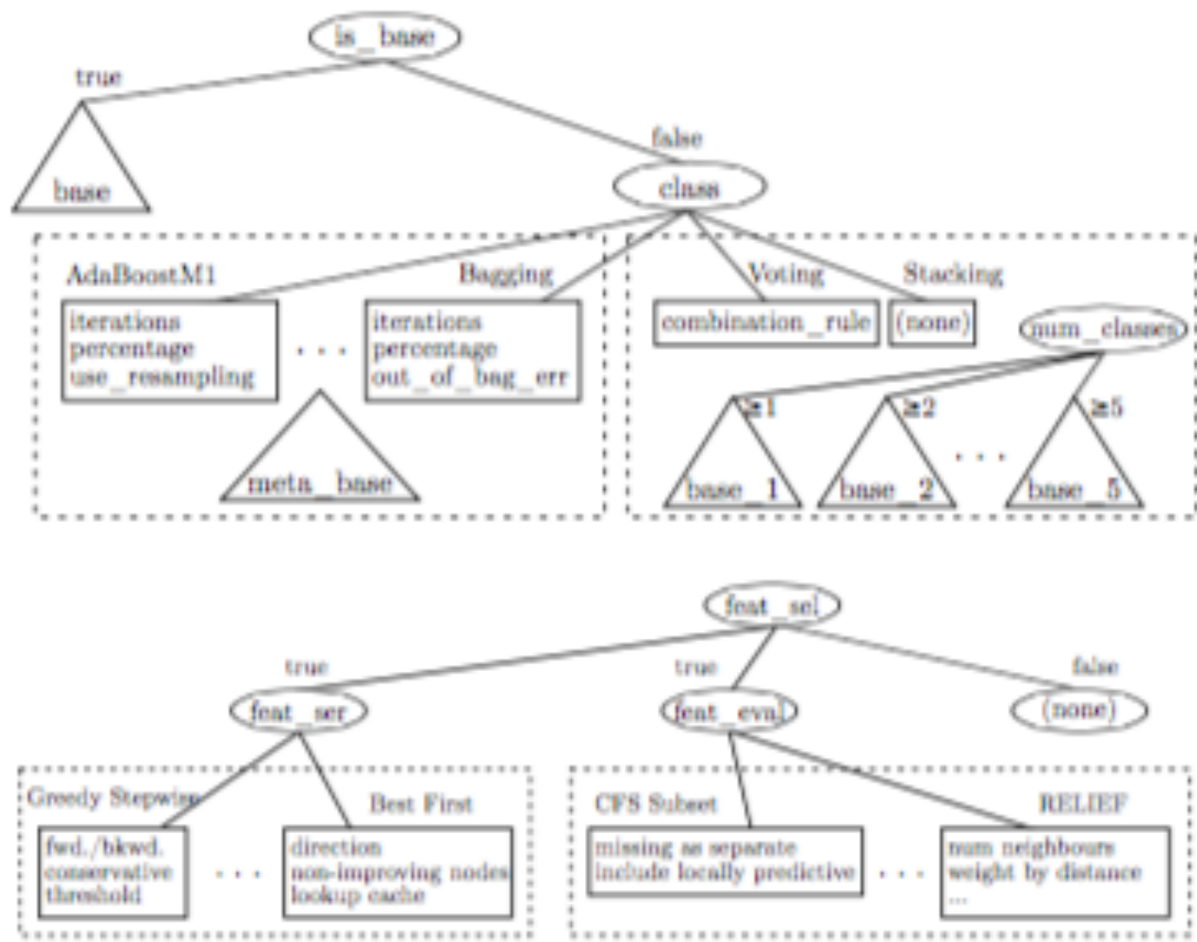


Figure 1: Auto-WEKA's parameter space. Top: first top level Boolean, concerning Auto-WEKA's classification methods. The triangular items represent a parameter that selects one of the 27 base classifiers, and adds conditional classifier hyperparameters accordingly. Bottom: second top level Boolean, concerning Auto-WEKA's feature selection methods.

[2] C. Thornton, F. Hutter, H. H. Hoos, K. Leyton-Brown. Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms. KDD, 2013.

Reference

[1] AutoML workshop @ ICML'15 Official Site. https://sites.google.com/site/automlwsicml15/

[2] C. Thornton, F. Hutter, H. H. Hoos, K. Leyton-Brown. Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms. KDD, 2013.

[3] T. Domhan, T. Springenberg, F. Hutter. Extrapolating Learning Curves of Deep Neural Networks. ICML 2014 AutoML Workshop, 2014.

[4] I. Guyon, et al. Design of the 2015 ChaLearn AutoML Challenge. 2015.

[5] J. Snoek, H. Larochelle, R. P. Adams. Practical Bayesian Optimization of Machine Learning Algorithms. NIPS, 2012.

[6] F. Hutter. Bayesian Optimization for More Automatic Machine Learning. ECAI 2014 Meta-Learning and Algorithm Selection Workshop, 2014.

[7] E. Brochu, V. M. Cora, N. de Freitas. A tutorial on Bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning. Technical Report UBC TR-2009-23 and arXiv: 1012.2599v1, 2009.

[8] J. Močkus. Application of Bayesian approach to numerical methods of global and stochastic optimization. Journal of Global Optimization, 4(4):347-365, 1994.

[9] H. J. Kushner. A new method of locating the maximum of an arbitrary multipeak curve in the presence of noise. Journal of Basic Engineering, 86:97-106, 1964.

[10] J. Močkus, V. Tiesis, and A. Žilinskas. The application of Bayesian methods for seeking the extremum. Towards Global Optimization, 2:117-129, 1978.

[11] D. R. Jones, M. Schonlau, and W. J. Welch. Efficient global optimization of of expensive black-box functions. Journal of Global Optimization, 13(4):455-492, 1998.

[12] N. Srinivas, A. Krause, S. M. Kakade, and M. Seeger. Gaussian process optimization in the bandit setting: No regret and experimental design. ICML, 2010.

[13] M. Feurer, J. T. Springenberg, and F. Hutter. Using Meta-Learning to Initialize Bayesian Optimization of Hyperparameters. ECAI workshop on Metalearning and Algorithm Selection (MetaSel), 2014.

[14] F. Hutter, H. H. Hoos, and K. Leyton-Brown. Sequential Model-Based Optimization for General Algorithm Configuration. Learning and Intelligent Optimization, 507-523, 2011.

[15] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl. Algorithms for Hyper-Parameter Optimization. NIPS, 2011.